ANNA UNIVERSITY, CEG CAMPUS, CHENNAI

[Department Name]

ADVANCED DATA STRUCTURES AND ALGORITHMS

Mini Project Report

ACADEMIC RECORDS INDEXER: A HIGH-PERFORMANCE BINARY SEARCH TREE ARCHITECTURE FOR EDUCATIONAL DATA MANAGEMENT

Submitted by

DEEPTANSHU KUMAR

Roll No. 2026166028

M.Tech. Artificial Intelligence and Data Science

Faculty: Dr. M. Vijayalakshmi

November 2026

Abstract

Managing academic records in high-throughput graduate programs requires data structures optimized for rapid retrieval, sequential output, and dynamic memory allocation. This project introduces the Academic Records Indexer, a high-performance database utility built entirely in C utilizing a Binary Search Tree (BST) architecture. Designed to process granular evaluation metrics, specifically tracking Continuous Internal Assessments (CIA 1, CIA 2) and End-Semester examination marks, the system utilizes student roll numbers as the primary sorting key. Through a streamlined command-line interface, administrators can seamlessly perform real-time data entry, instant grade updates, and targeted record deletions without disrupting the underlying hierarchical structure.

The implementation guarantees an O(log N) average-case time complexity for both record insertion and targeted student lookups, vastly outperforming linear data models. Furthermore, the system employs an in-order tree traversal algorithm to automatically generate sequentially sorted academic marksheets in O(N) time. To maximize memory efficiency and maintain clean logic, standard integer flags evaluate student eligibility and academic standing. Additionally, the architecture incorporates robust edge-case handling to strictly prevent duplicate roll number entries and utilizes systematic memory deallocation protocols to eliminate memory leaks during program termination.

To validate structural efficiency, the project incorporates a rigorous benchmarking suite utilizing the <time.h> library, demonstrating the exponentially lower CPU execution cycles required for BST lookups against large-scale simulated datasets compared to traditional array-based searching: at one million records a BST lookup needed 25.7 key comparisons on average against 504,168 for a linear scan, a 943-fold reduction in measured CPU time. Ultimately, this utility bridges theoretical algorithmic concepts with practical administrative applications, delivering a scalable, resource-light framework that can be readily adapted into broader university evaluation pipelines.

Table of Contents

Abstract 2
Table of Contents 3
1. Introduction 4
  1.1 Background 4
  1.2 Objectives 4
  1.3 Scope 4
2. System Design 4
  2.1 Core data structure 4
  2.2 Comparison with alternative structures 5
  2.3 Algorithms 5
  2.4 Complexity summary 7
3. Implementation 7
  3.1 Program structure 7
  3.2 Input validation and robustness 8
  3.3 Build and run 8
4. Testing and Demonstration 8
  4.1 Tree statistics after loading 8
  4.2 Marksheet by in-order traversal 8
  4.3 Search, duplicate rejection, insertion and update 9
  4.4 Deletion of a leaf and of the root 9
  4.5 Persistence and deallocation 9
5. Benchmark Results 10
  5.1 Method 10
  5.2 Results 10
  5.3 Worst case 10
6. Conclusion and Future Work 11
References 11
Appendix A - Source code: academic_indexer.c 12
Appendix B - Source code: benchmark.c 18
Appendix C - Sample data (data/students.csv) 21

(Right-click and choose Update Field in Word to refresh page numbers.)

1. Introduction

1.1 Background

Every academic department maintains, for each enrolled student, a set of evaluation marks that is created at admission, modified repeatedly during the semester as internal assessments and the end-semester examination are graded, and finally consolidated into a marksheet. Two access patterns dominate: an administrator must retrieve one student's record instantly when a query arrives, and the office must print the whole class in ascending roll-number order for notice boards and result processing. The data set is dynamic, so insertions, corrections and deletions interleave with the queries.

A linear structure cannot serve both patterns efficiently. An unsorted array or linked list answers a query only after scanning up to N records; a sorted array answers in O(log N) with binary search but needs O(N) element shifts for every insertion or deletion. A Binary Search Tree (BST) keeps the records ordered at all times while supporting insertion, search and deletion in O(log N) average time, and an in-order traversal yields the sorted listing in O(N) without any sorting step. This makes the BST a natural index for an academic records manager.

1.2 Objectives

1. Design a C struct that acts as a BST node holding the roll number as the key, CIA 1, CIA 2 and end-semester marks, integer status flags and left and right pointers.
2. Implement insertion, lookup, update and deletion keyed on the roll number with O(log N) average complexity, rejecting duplicate roll numbers.
3. Generate the class marksheet in ascending roll order with an in-order traversal in O(N) time, including computed totals, grades and eligibility flags.
4. Provide a command-line interface with CSV load and save, robust input validation and systematic deallocation of all nodes at exit.
5. Benchmark BST lookups against array-based searching on simulated datasets of up to one million records using the <time.h> library.

1.3 Scope

The project is a single-user console utility holding records in main memory. Persistence is a plain CSV file. Balancing, concurrency and on-disk storage are outside the scope and are discussed as future work.

2. System Design

2.1 Core data structure

The record and the tree node are the same struct (Listing 2.1). The roll number is the primary key; all keys in a node's left sub-tree are smaller and all keys in its right sub-tree are larger. The

two status flags are plain integers, as required by the specification, so no external boolean header is needed and the struct compiles with any C compiler.

Listing 2.1 The BST node

```
typedef struct StudentNode {
    long  roll_number;              /* primary key                       */
    char  name[NAME_LEN];
    float cia1;                     /* Continuous Internal Assessment 1  */
    float cia2;                     /* Continuous Internal Assessment 2  */
    float end_sem;                  /* End-Semester examination          */
    int   is_eligible;              /* 1 = eligible to write end-sem     */
    int   has_arrears;              /* 1 = has arrear in this course     */
    struct StudentNode *left;       /* smaller roll numbers              */
    struct StudentNode *right;      /* larger roll numbers               */
} StudentNode;
```

Marks are stored as entered (CIA 1 and CIA 2 out of 50, end-semester out of 100). Derived quantities are computed from them by small helper functions: internal marks out of 40, external marks out of 60, the total out of 100 and the letter grade. The flags are recomputed by refresh_flags whenever marks are inserted or updated: is_eligible is set when the internal percentage reaches 40, and has_arrears when the total is below 50 or the student is not eligible. All thresholds are #define constants so the program can be adapted to another regulation by editing one block.

2.2 Comparison with alternative structures

Structure	Insert	Search	Delete	Sorted listing
Unsorted array	O(1)	O(N)	O(N)	O(N log N), sort first
Sorted array	O(N) shift	O(log N)	O(N) shift	O(N)
Linked list	O(1), or O(N) if kept sorted	O(N)	O(N)	O(N) if kept sorted
Hash table	O(1) average	O(1) average	O(1) average	O(N log N), sort first
BST (this project)	O(log N) average	O(log N) average	O(log N) average	O(N) in-order

Table 2.1 Time complexity of candidate structures

Only the BST is logarithmic for all three update operations and keeps the data ordered. A hash table gives faster single lookups but has no notion of order, so every marksheet would require an additional O(N log N) sort.

2.3 Algorithms

Insertion. Starting at the root, the incoming roll number is compared with the current node: a smaller key moves to the left child, a larger key to the right. The walk stops at a NULL link, where the new node is attached as a leaf, or at an equal key, in which case the insertion is refused as a duplicate. The implementation walks with a pointer-to-pointer and needs no recursion. The cost is the number of levels visited, bounded by the height h of the tree.

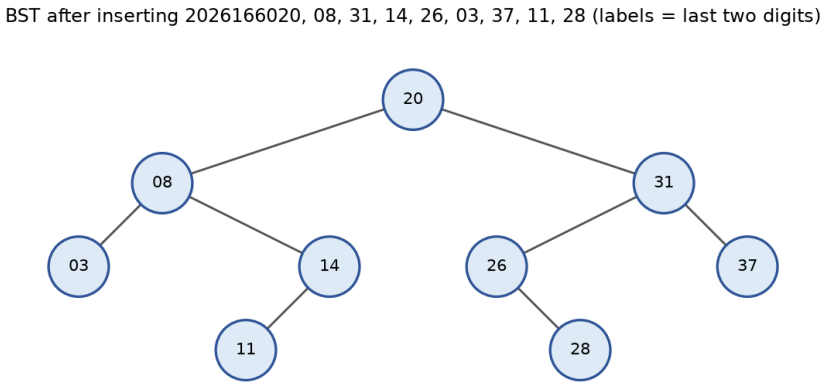


Figure 2.1 BST after inserting the first nine records of the sample file

Lookup. The search follows the same comparisons. Each comparison discards an entire sub-tree, so the number of candidates roughly halves at every step; the search ends when the key is found or a NULL link is reached. Figure 2.2 shows the path to roll number 2026166028, which takes four comparisons in a nine-record tree. The program counts comparisons during every search and reports them, so the logarithmic behaviour can be observed directly.

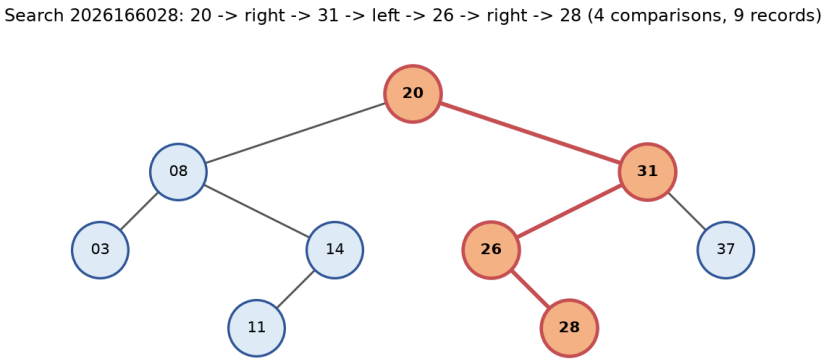


Figure 2.2 Search path for 2026166028

Deletion. Three cases arise (Figure 2.3). A leaf is unlinked from its parent and freed. A node with one child is replaced by that child. A node with two children receives a copy of its in-order successor, the minimum of its right sub-tree, after which the successor, which has at most one child, is deleted recursively. The recursive formulation returns the possibly new sub-tree root so that parent links are rewired on the way back up.

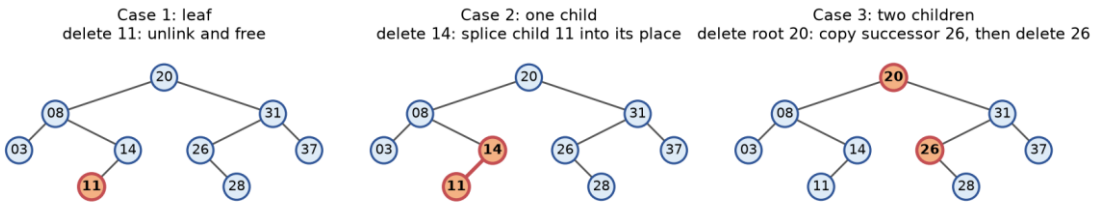


Figure 2.3 The three deletion cases

In-order traversal. Visiting the left sub-tree, then the node, then the right sub-tree enumerates keys in ascending order. The marksheet, the CSV export and the record count all use this

traversal; each node is visited exactly once, so the cost is O(N) with O(h) stack space. A pre-order traversal with indentation is also provided to display the actual shape of the tree during the demonstration.

Deallocation. At exit a post-order traversal frees the left and right sub-trees before the node itself, so no node is freed before its children are reached and no pointer is dereferenced after being freed. The program reports the number of nodes released.

2.4 Complexity summary

Operation	Average	Worst case	Note
Insert	O(log N)	O(N)	worst case when keys arrive sorted
Search	O(log N)	O(N)	comparisons = depth of the key
Update	O(log N)	O(N)	search then constant work
Delete	O(log N)	O(N)	successor search is also O(h)
Marksheet	O(N)	O(N)	in-order traversal
Free all	O(N)	O(N)	post-order traversal
Space	O(N)	O(N)	80 bytes per record on a 64-bit build

Table 2.2 Complexity of the implemented operations (h = height, about 2 log2 N for random insertion order)

3. Implementation

3.1 Program structure

The program is a single C file of about 350 lines, organised in six sections: the data structure and academic helpers, BST primitives, presentation helpers, input helpers, CSV load and save, and the menu actions with main. It compiles without warnings under gcc -O2 -Wall -Wextra and uses only the standard library.

Menu	Action	Implementation
1	Insert student record	bst_search to reject duplicates, create_node, bst_insert; flags and grade reported
2	Search by roll number	bst_search; prints the record and the comparison count
3	Update marks	bst_search, overwrite marks, refresh_flags
4	Delete record	bst_delete with the three cases; reports children, new count and height
5	Class marksheet	inorder_print with a formatted table
6	Tree statistics	count, height versus balanced minimum, root, min, max, pre-order shape
7	Load CSV	fgets and sscanf per line; duplicates and malformed lines counted and skipped
8	Save CSV	inorder_save writes the records in ascending order
0	Exit	bst_free in post-order; reports nodes released

Table 3.1 Menu actions

3.2 Input validation and robustness

Every scanf result is checked; on failure the remainder of the line is discarded and the menu re-prompts instead of looping on the bad token. Marks are range-checked (0 to 50 for CIA, 0 to 100 for end-semester) before a record is created or modified. Names are read with fgets and truncated to the buffer size. When input is redirected from a file, the program echoes the values it reads so that scripted demonstrations produce a readable transcript.

3.3 Build and run

```
gcc -O2 -Wall -Wextra -o academic_indexer src/academic_indexer.c
gcc -O2 -Wall -Wextra -o benchmark src/benchmark.c
./academic_indexer data/students.csv                # interactive, preloaded
./academic_indexer data/students.csv < demo/demo_input.txt   # scripted demo
cd results && ../benchmark                           # benchmark suite
```

4. Testing and Demonstration

A sample class of 25 records (data/students.csv) is used for all functional tests. Its insertion order is deliberately scrambled so that the tree stays close to the balanced minimum height. The complete scripted session is in results/demo_transcript.txt; the excerpts below are taken from it verbatim.

4.1 Tree statistics after loading

```
  records      : 25
  tree height  : 5  (balanced minimum would be 5)
  root         : 2026166020 (Karthik Raman)
  smallest roll: 2026166001 (Abdul Rahman)   largest roll: 2026166040 (Yuvraj Singh)
  worst-case comparisons for a lookup = height = 5  (linear scan would need up to 25)
```

With 25 records the tree has height 5, which is the minimum possible for 25 nodes, so a lookup needs at most 5 comparisons where a linear scan could need 25.

4.2 Marksheet by in-order traversal

```
  CLASS MARKSHEET - in-order traversal, ascending roll number (25 records)
+------------+----------------------+------+------+------+-------+-------+-------+-----+------+
| Roll No    | Name                 | CIA1 | CIA2 | EndS | Int40 | Ext60 | Total | Grd | Flag |
+------------+----------------------+------+------+------+-------+-------+-------+-----+------+
| 2026166001 | Abdul Rahman         |   37 |   39 |   66 |  30.4 |  39.6 |  70.0 | B+  | E-   |
| 2026166002 | Ajay Menon           |   35 |   33 |   69 |  27.2 |  41.4 |  68.6 | B+  | E-   |
| 2026166003 | Anjali Nair          |   47 |   49 |   93 |  38.4 |  55.8 |  94.2 | O   | E-   |
| 2026166005 | Fathima Begum        |   41 |   44 |   79 |  34.0 |  47.4 |  81.4 | A+  | E-   |
| 2026166007 | Keerthana M          |   46 |   48 |   91 |  37.6 |  54.6 |  92.2 | O   | E-   |
| 2026166008 | Priya Lakshmi        |   45 |   47 |   88 |  36.8 |  52.8 |  89.6 | A+  | E-   |
| ...        | ...                  |  ... |  ... |  ... |   ... |   ... |   ... | ... | ...  |
| 2026166040 | Yuvraj Singh         |   40 |   41 |   74 |  32.4 |  44.4 |  76.8 | A   | E-   |
+------------+----------------------+------+------+------+-------+-------+-------+-----+------+
  Flag: E = eligible for end-sem (internal >= 40%), A = arrear (total < 50 or not eligible)
```

Rows appear in ascending roll order although they were inserted in scrambled order. The Flag column shows E for exam-eligible and A for arrear: Rahul Verma (2026166026) and Manoj

Prabhu (2026166033) have internal marks below 40 percent and are therefore not eligible; Suresh Pandian and Harish Babu are eligible but failed the total.

4.3 Search, duplicate rejection, insertion and update

```
Roll number to find: 2026166028
found in 4 comparisons out of 25 records
Roll number to find: 2026166099
not found (5 comparisons, tree holds 25 records)


Roll number      : 2026166028
! roll 2026166028 already exists - duplicate rejected


Roll number      : 2026166010
Name             : Ishwarya Venkat
CIA 1 (0-50)     : 44
CIA 2 (0-50)     : 43
End-sem (0-100)  : 82
inserted 2026166010 (Ishwarya Venkat): total 84.0, grade A+, eligible=1, arrears=0


current: CIA1 15  CIA2 18  End-sem 40  (total 37.2, grade U)
new CIA 1 (0-50)    : 30
new CIA 2 (0-50)    : 32
new End-sem (0-100) : 55
updated: total 57.8, grade B, eligible=1, arrears=0
```

The author's own record is found in four comparisons; a non-existent roll number is rejected after five, the height of the tree. Re-inserting an existing roll number is refused. A new record and an update both recompute the total, grade and flags: after the update Rahul Verma becomes eligible, his arrear is cleared and his grade changes from U to B.

4.4 Deletion of a leaf and of the root

```
deleted 2026166033 (node had 0 children); tree now holds 25 records, height 5
deleted 2026166020 (node had 2 children); tree now holds 24 records, height 5


records      : 24
tree height  : 5  (balanced minimum would be 5)
root         : 2026166023 (Gokul Krishnan)
smallest roll: 2026166001 (Abdul Rahman)   largest roll: 2026166040 (Yuvraj Singh)
```

Deleting the leaf 2026166033 and then the root 2026166020 (two children) leaves a valid tree of 24 records whose new root is the in-order successor 2026166023; the height is unchanged and the subsequent marksheet is still sorted.

4.5 Persistence and deallocation

```
saved 24 records to results/students_sorted_out.csv (in ascending roll order)
freed 24 nodes - all memory released. Goodbye.
```

Option 8 writes the records in ascending order and option 0 frees all 24 nodes before exit. Invalid input was tested separately: a non-numeric menu choice and an out-of-range mark are both reported and the program continues.

5. Benchmark Results

5.1 Method

A separate program (src/benchmark.c) generates N unique roll numbers, shuffles them, builds a BST by insertion and also stores them in an unsorted array and in a sorted array. It then looks up 1,000 randomly chosen existing keys with three methods: BST search, linear scan of the unsorted array and binary search of the sorted array. Because a single lookup completes in far less than one tick of clock(), each method repeats the 1,000 lookups until at least 0.25 s of CPU time has elapsed and the average time per lookup is reported. The exact number of key comparisons is counted as a machine-independent measure. Finally the worst case is reproduced by inserting 10,000 keys in ascending order.

5.2 Results

N	BST height	Build (ms)	BST us/lookup	BST cmp	Linear us/lookup	Linear cmp	Binary us/lookup	Binary cmp	Speed-up
1,000	27	0	0.015	11.4	0.113	487	0.020	8.9	8x
10,000	32	1	0.033	16.9	1.092	4,966	0.033	12.4	33x
100,000	41	14	0.058	21.2	10.792	49,060	0.053	15.7	186x
1,000,000	47	514	0.121	25.7	114.333	504,168	0.079	19.0	943x

Table 5.1 Lookup cost by method (speed-up = linear time / BST time)

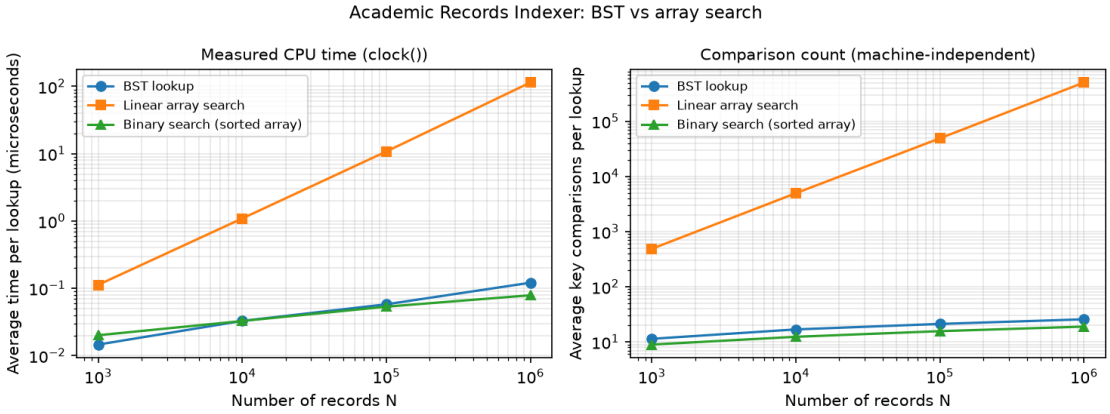


Figure 5.1 Average time and comparisons per lookup against N (log-log axes)

Each ten-fold increase in N multiplies the work of the linear scan by ten, whereas the BST adds only four or five comparisons: the comparison count grows from 11.4 at N = 1,000 to 25.7 at N = 1,000,000, following the tree height (27 to 47). At one million records a BST lookup took 0.121 microseconds against 114.3 for the linear scan, a 943-fold reduction in CPU time. Binary search on a sorted array is marginally faster per lookup than the BST because the array is contiguous and perfectly balanced, but it cannot insert or delete in logarithmic time, which the records manager requires.

5.3 Worst case

Inserting 10,000 keys in ascending order produced a tree of height 10000, a linked list, in which a lookup needed 5053 comparisons and 15.8 microseconds on average: linear behaviour.

Random insertion order of the same 10,000 keys gave height 32 and 16.9 comparisons. The sample CSV is therefore scrambled, and a production deployment would use a self-balancing AVL or Red-Black tree.

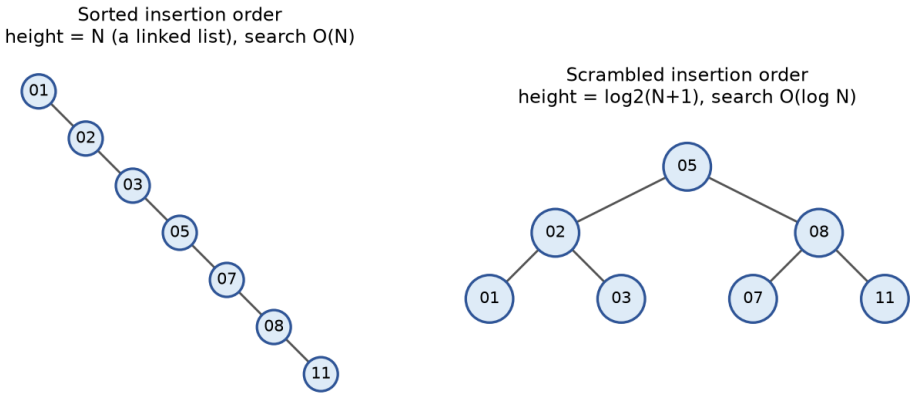


Figure 5.2 Sorted insertion order degenerates the tree; scrambled order keeps it balanced

6. Conclusion and Future Work

The Academic Records Indexer meets every requirement of the specification: a BST node holding the roll number, CIA and end-semester marks, integer flags and child pointers; O(log N) average insertion, lookup, update and deletion with duplicate rejection; an O(N) in-order marksheet; CSV persistence; validated input; and complete deallocation at exit. The benchmark confirms the complexity claims empirically, with a 943-fold lookup speed-up over linear search at one million records and comparison counts that grow with log N.

The main limitation of a plain BST is its dependence on insertion order. Future work would replace it with an AVL or Red-Black tree to guarantee O(log N) in the worst case, add secondary indexes by name and department, store nodes in disk pages as a B+ tree for a university-scale database, and add range queries (all roll numbers between two values), which the ordered structure already supports naturally.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 12 (Binary Search Trees).

[2] M. A. Weiss, Data Structures and Algorithm Analysis in C, 2nd ed. Reading, MA, USA: Addison-Wesley, 1997, ch. 4.

[3] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[4] ISO/IEC 9899:2018, Information technology - Programming languages - C, clause 7.27 (Date and time <time.h>).

[5] G. M. Adelson-Velsky and E. M. Landis, "An algorithm for the organization of information," Soviet Mathematics Doklady, vol. 3, pp. 1259-1263, 1962.

Appendix A - Source code: academic_indexer.c

```
/*
 * Academic Records Indexer
 * A Binary Search Tree (BST) based student-record manager written in C.
 *
 * Course   : Advanced Data Structures and Algorithms (Mini Project)
 * Student  : Deeptanshu Kumar, Roll No. 2026166028, M.Tech AI & DS
 * College  : Anna University, CEG Campus, Chennai
 * Faculty  : Dr. M. Vijayalakshmi
 *
 * Key      : roll_number (unique, ascending order in the tree)
 * Payload  : CIA 1, CIA 2 (each out of 50), End-Semester (out of 100)
 * Flags    : is_eligible (internal >= 40 %), has_arrears (total < 50 or not eligible)
 *
 * Operations (menu):   insert, search, update, delete, in-order marksheet,
 *                      statistics, load/save CSV, free all memory on exit.
 * Complexity:          insert / search / delete   O(h)  = O(log N) average
 *                      in-order marksheet         O(N)
 *
 * Build:   gcc -O2 -Wall -o academic_indexer academic_indexer.c
 * Run:     ./academic_indexer                 (interactive)
 *          ./academic_indexer < demo_input.txt (scripted demo)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef _WIN32
#  include <io.h>
#  define STDIN_IS_TTY() _isatty(_fileno(stdin))
#else
#  include <unistd.h>
#  define STDIN_IS_TTY() isatty(fileno(stdin))
#endif


/* ---------- configurable academic thresholds ---------- */
#define CIA_MAX            50.0f   /* each CIA is out of 50               */
#define ENDSEM_MAX        100.0f   /* end-semester paper is out of 100    */
#define INTERNAL_WEIGHT     0.40f  /* internal contributes 40 % of total  */
#define EXTERNAL_WEIGHT     0.60f  /* end-sem contributes 60 % of total   */
#define ELIGIBLE_INTERNAL  40.0f   /* min internal % to be exam-eligible  */
#define PASS_TOTAL         50.0f   /* min total (out of 100) to pass      */
#define NAME_LEN           40


/* ======================================================
 *  1. Core data structure : the BST node
 * ====================================================== */
typedef struct StudentNode {
    long  roll_number;              /* primary key                       */
    char  name[NAME_LEN];
    float cia1;                     /* Continuous Internal Assessment 1  */
    float cia2;                     /* Continuous Internal Assessment 2  */
    float end_sem;                  /* End-Semester examination          */
    int   is_eligible;              /* 1 = eligible to write end-sem     */
    int   has_arrears;              /* 1 = has arrear in this course     */
    struct StudentNode *left;       /* smaller roll numbers              */
    struct StudentNode *right;      /* larger roll numbers               */
} StudentNode;


/* global counters used for the statistics screen */
static long g_comparisons = 0;      /* key comparisons in the last search */


/* ---------- derived academic values ---------- */
static float internal_marks(const StudentNode *s)          /* out of 40 */
{
    return (s->cia1 + s->cia2) / (2.0f * CIA_MAX) * 100.0f * INTERNAL_WEIGHT;
}
static float external_marks(const StudentNode *s)          /* out of 60 */
{
    return s->end_sem / ENDSEM_MAX * 100.0f * EXTERNAL_WEIGHT;
}
static float total_marks(const StudentNode *s)             /* out of 100 */
{
    return internal_marks(s) + external_marks(s);
}
static const char *letter_grade(const StudentNode *s)
{
    float t = total_marks(s);
    if (s->has_arrears) return "U";
    if (t >= 91) return "O";
    if (t >= 81) return "A+";
    if (t >= 71) return "A";
    if (t >= 61) return "B+";
    if (t >= 50) return "B";
```

```
    return "U";
}
/* recompute the integer status flags from the marks */
static void refresh_flags(StudentNode *s)
{
    float internal_pct = (s->cia1 + s->cia2) / (2.0f * CIA_MAX) * 100.0f;
    s->is_eligible = (internal_pct >= ELIGIBLE_INTERNAL) ? 1 : 0;
    s->has_arrears = (!s->is_eligible || total_marks(s) < PASS_TOTAL) ? 1 : 0;
}


/* ====================================================
 *  2. BST primitives
 * ==================================================== */
static StudentNode *create_node(long roll, const char *name,
                                float c1, float c2, float es)
{
    StudentNode *n = (StudentNode *)malloc(sizeof(StudentNode));
    if (!n) { fprintf(stderr, "Out of memory\n"); exit(EXIT_FAILURE); }
    n->roll_number = roll;
    strncpy(n->name, name, NAME_LEN - 1);
    n->name[NAME_LEN - 1] = '\0';
    n->cia1 = c1; n->cia2 = c2; n->end_sem = es;
    n->left = n->right = NULL;
    refresh_flags(n);
    return n;
}

/* Insertion - O(h). Returns 1 on success, 0 if the roll number already exists. */
static int bst_insert(StudentNode **root, StudentNode *node)
{
    StudentNode **cur = root;          /* pointer-to-pointer walk: no recursion */
    while (*cur) {
        if (node->roll_number < (*cur)->roll_number)       cur = &(*cur)->left;
        else if (node->roll_number > (*cur)->roll_number)  cur = &(*cur)->right;
        else return 0;                                     /* duplicate key   */
    }
    *cur = node;
    return 1;
}


/* Rapid lookup - O(h). Each step discards one entire sub-tree. */
static StudentNode *bst_search(StudentNode *root, long roll)
{
    g_comparisons = 0;
    while (root) {
        g_comparisons++;
        if (roll == root->roll_number) return root;
        root = (roll < root->roll_number) ? root->left : root->right;
    }
    return NULL;
}


static StudentNode *bst_min(StudentNode *n)
{
    while (n && n->left) n = n->left;
    return n;
}
static StudentNode *bst_max(StudentNode *n)
{
    while (n && n->right) n = n->right;
    return n;
}


/* Deletion - O(h). Handles the three classic cases:
 *   leaf, one child, two children (replace by in-order successor). */
static StudentNode *bst_delete(StudentNode *root, long roll, int *deleted)
{
    if (!root) return NULL;
    if (roll < root->roll_number)
        root->left = bst_delete(root->left, roll, deleted);
    else if (roll > root->roll_number)
        root->right = bst_delete(root->right, roll, deleted);
    else {
        *deleted = 1;
        if (!root->left) {                      /* 0 or 1 child (right)  */
            StudentNode *r = root->right; free(root); return r;
        }
        if (!root->right) {                     /* 1 child (left)        */
            StudentNode *l = root->left; free(root); return l;
        }
        /* two children: copy in-order successor's data, then delete it */
        StudentNode *succ = bst_min(root->right);
        root->roll_number = succ->roll_number;
        strcpy(root->name, succ->name);
        root->cia1 = succ->cia1; root->cia2 = succ->cia2; root->end_sem = succ->end_sem;
```

```
        root->is_eligible = succ->is_eligible; root->has_arrears = succ->has_arrears;
        int dummy = 0;
        root->right = bst_delete(root->right, succ->roll_number, &dummy);
    }
    return root;
}

static int  bst_count(StudentNode *n)  { return n ? 1 + bst_count(n->left) + bst_count(n->right) : 0; }
static int  bst_height(StudentNode *n)
{
    if (!n) return 0;
    int l = bst_height(n->left), r = bst_height(n->right);
    return 1 + (l > r ? l : r);
}

/* Systematic deallocation - post-order so children are freed before parent */
static void bst_free(StudentNode *n)
{
    if (!n) return;
    bst_free(n->left);
    bst_free(n->right);
    free(n);
}

/* ====================================================
 *  3. Presentation helpers
 * ==================================================== */
static void print_header(void)
{
    printf("+------------+----------------------+------+------+------+-------+-------+-------+-----+------+\n");
    printf("| Roll No    | Name                 | CIA1 | CIA2 | EndS | Int40 | Ext60 | Total | Grd | Flag |\n");
    printf("+------------+----------------------+------+------+------+-------+-------+-------+-----+------+\n");
}
static void print_row(const StudentNode *s)
{
    printf("| %-10ld | %-20s | %4.0f | %4.0f | %4.0f | %5.1f | %5.1f | %5.1f | %-3s | %c%c   |\n",
           s->roll_number, s->name, s->cia1, s->cia2, s->end_sem,
           internal_marks(s), external_marks(s), total_marks(s), letter_grade(s),
           s->is_eligible ? 'E' : '-', s->has_arrears ? 'A' : '-');
}
static void print_footer(void)
{
    printf("+------------+----------------------+------+------+------+-------+-------+-------+-----+------+\n");
    printf("  Flag: E = eligible for end-sem (internal >= %.0f%%), A = arrear (total < %.0f or not eligible)\n",
           ELIGIBLE_INTERNAL, PASS_TOTAL);
}

/* In-order traversal (Left - Root - Right) - O(N) - ascending roll numbers */
static void inorder_print(const StudentNode *n)
{
    if (!n) return;
    inorder_print(n->left);
    print_row(n);
    inorder_print(n->right);
}
/* Pre-order shows the actual tree shape (root first) - useful in the demo */
static void preorder_print(const StudentNode *n, int depth)
{
    if (!n) return;
    printf("%*s%ld (%s)\n", depth * 4, "", n->roll_number, n->name);
    preorder_print(n->left, depth + 1);
    preorder_print(n->right, depth + 1);
}
static void inorder_save(const StudentNode *n, FILE *fp)
{
    if (!n) return;
    inorder_save(n->left, fp);
    fprintf(fp, "%ld,%s,%.1f,%.1f,%.1f\n", n->roll_number, n->name, n->cia1, n->cia2, n->end_sem);
    inorder_save(n->right, fp);
}

/* ====================================================
 *  4. Input helpers (robust against bad input)
 * ==================================================== */
static void flush_line(void) { int c; while ((c = getchar()) != '\n' && c != EOF) {} }

static int read_long(const char *prompt, long *out)
{
    printf("%s", prompt);
    if (scanf("%ld", out) != 1) { flush_line(); printf("  ! invalid number\n"); return 0; }
    flush_line();
    if (!STDIN_IS_TTY()) printf("%ld\n", *out);     /* echo when input is piped */
    return 1;
}
```

```
static int read_float_range(const char *prompt, float lo, float hi, float *out)
{
    printf("%s", prompt);
    if (scanf("%f", out) != 1) { flush_line(); printf("  ! invalid number\n"); return 0; }
    flush_line();
    if (!STDIN_IS_TTY()) printf("%g\n", *out);
    if (*out < lo || *out > hi) { printf("  ! must be between %.0f and %.0f\n", lo, hi); return 0; }
    return 1;
}
static void read_name(const char *prompt, char *buf)
{
    printf("%s", prompt);
    if (!fgets(buf, NAME_LEN, stdin)) { buf[0] = '\0'; return; }
    buf[strcspn(buf, "\r\n")] = '\0';
    if (buf[0] == '\0') strcpy(buf, "(no name)");
    if (!STDIN_IS_TTY()) printf("%s\n", buf);
}


/* ====================================================
 *  5. CSV load / save
 * ==================================================== */
static int load_csv(StudentNode **root, const char *path)
{
    FILE *fp = fopen(path, "r");
    if (!fp) { printf("  ! cannot open %s\n", path); return 0; }
    char line[256]; int added = 0, dup = 0, bad = 0;
    while (fgets(line, sizeof line, fp)) {
        long roll; char name[NAME_LEN]; float c1, c2, es;
        if (line[0] == '#' || line[0] == '\n') continue;
        if (sscanf(line, "%ld,%39[^,],%f,%f,%f", &roll, name, &c1, &c2, &es) != 5) { bad++; continue; }
        if (bst_insert(root, create_node(roll, name, c1, c2, es))) added++; else dup++;
    }
    fclose(fp);
    printf("  loaded %d records from %s (%d duplicates skipped, %d malformed lines)\n", added, path, dup, bad);
    return added;
}
static void save_csv(StudentNode *root, const char *path)
{
    FILE *fp = fopen(path, "w");
    if (!fp) { printf("  ! cannot write %s\n", path); return; }
    fprintf(fp, "# roll_number,name,cia1,cia2,end_sem\n");
    inorder_save(root, fp);
    fclose(fp);
    printf("  saved %d records to %s (in ascending roll order)\n", bst_count(root), path);
}


/* ====================================================
 *  6. Menu actions
 * ==================================================== */
static void action_insert(StudentNode **root)
{
    long roll; char name[NAME_LEN]; float c1, c2, es;
    if (!read_long("  Roll number      : ", &roll)) return;
    if (bst_search(*root, roll)) { printf("  ! roll %ld already exists - duplicate rejected\n", roll); return; }
    read_name("  Name             : ", name);
    if (!read_float_range("  CIA 1 (0-50)     : ", 0, CIA_MAX, &c1)) return;
    if (!read_float_range("  CIA 2 (0-50)     : ", 0, CIA_MAX, &c2)) return;
    if (!read_float_range("  End-sem (0-100)  : ", 0, ENDSEM_MAX, &es)) return;
    StudentNode *n = create_node(roll, name, c1, c2, es);
    if (bst_insert(root, n)) {
        printf("  inserted %ld (%s): total %.1f, grade %s, eligible=%d, arrears=%d\n",
               roll, name, total_marks(n), letter_grade(n), n->is_eligible, n->has_arrears);
    } else { free(n); printf("  ! duplicate rejected\n"); }
}


static void action_search(StudentNode *root)
{
    long roll;
    if (!read_long("  Roll number to find: ", &roll)) return;
    StudentNode *s = bst_search(root, roll);
    if (!s) { printf("  not found (%ld comparisons, tree holds %d records)\n", g_comparisons, bst_count(root)); retur
n; }
    print_header(); print_row(s); print_footer();
    printf("  found in %ld comparisons out of %d records\n", g_comparisons, bst_count(root));
}


static void action_update(StudentNode *root)
{
    long roll; float c1, c2, es;
    if (!read_long("  Roll number to update: ", &roll)) return;
    StudentNode *s = bst_search(root, roll);
    if (!s) { printf("  not found\n"); return; }
    printf("  current: CIA1 %.0f  CIA2 %.0f  End-sem %.0f  (total %.1f, grade %s)\n",
           s->cia1, s->cia2, s->end_sem, total_marks(s), letter_grade(s));
    if (!read_float_range("  new CIA 1 (0-50)    : ", 0, CIA_MAX, &c1)) return;
    if (!read_float_range("  new CIA 2 (0-50)    : ", 0, CIA_MAX, &c2)) return;
```

```
    if (!read_float_range("  new End-sem (0-100) : ", 0, ENDSEM_MAX, &es)) return;
    s->cia1 = c1; s->cia2 = c2; s->end_sem = es;
    refresh_flags(s);
    printf("  updated: total %.1f, grade %s, eligible=%d, arrears=%d\n",
           total_marks(s), letter_grade(s), s->is_eligible, s->has_arrears);
}

static void action_delete(StudentNode **root)
{
    long roll; int deleted = 0;
    if (!read_long("  Roll number to delete: ", &roll)) return;
    StudentNode *s = bst_search(*root, roll);
    if (!s) { printf("  not found\n"); return; }
    int children = (s->left != NULL) + (s->right != NULL);
    *root = bst_delete(*root, roll, &deleted);
    printf("  deleted %ld (node had %d child%s); tree now holds %d records, height %d\n",
           roll, children, children == 1 ? "" : "ren", bst_count(*root), bst_height(*root));
}

static void action_marksheet(StudentNode *root)
{
    int n = bst_count(root);
    if (!n) { printf("  (no records)\n"); return; }
    printf("\n  CLASS MARKSHEET - in-order traversal, ascending roll number (%d records)\n", n);
    print_header(); inorder_print(root); print_footer();
}

static void action_stats(StudentNode *root)
{
    int n = bst_count(root);
    if (!n) { printf("  (no records)\n"); return; }
    StudentNode *lo = bst_min(root), *hi = bst_max(root);
    int h = bst_height(root);
    /* optimal height of a perfectly balanced tree with n nodes = ceil(log2(n+1)) */
    int opt = 0; for (int x = n; x > 0; x >>= 1) opt++;
    printf("  records      : %d\n", n);
    printf("  tree height  : %d  (balanced minimum would be %d)\n", h, opt);
    printf("  root         : %ld (%s)\n", root->roll_number, root->name);
    printf("  smallest roll: %ld (%s)   largest roll: %ld (%s)\n",
           lo->roll_number, lo->name, hi->roll_number, hi->name);
    printf("  worst-case comparisons for a lookup = height = %d  (linear scan would need up to %d)\n", h, n);
    printf("\n  Tree shape (pre-order, indentation = depth):\n");
    preorder_print(root, 1);
}

static void show_menu(void)
{
    printf("\n==================== ACADEMIC RECORDS INDEXER (BST) ====================\n");
    printf(" 1. Insert student record        5. Print class marksheet (in-order)\n");
    printf(" 2. Search by roll number        6. Tree statistics / shape\n");
    printf(" 3. Update marks                 7. Load records from CSV\n");
    printf(" 4. Delete record                8. Save records to CSV\n");
    printf(" 0. Exit (free all memory)\n");
    printf("------------------------------------------------------------------------\n");
    printf(" choice: ");
}

int main(int argc, char **argv)
{
    StudentNode *root = NULL;
    if (argc > 1) load_csv(&root, argv[1]);     /* optional: preload a CSV */

    for (;;) {
        show_menu();
        int choice;
        if (scanf("%d", &choice) != 1) {
            if (feof(stdin)) { choice = 0; printf("0\n"); }
            else { flush_line(); printf("  ! enter a number 0-8\n"); continue; }
        } else {
            flush_line();
            printf("%d\n", choice);                /* echo so scripted demos read well */
        }
        switch (choice) {
            case 1: action_insert(&root);   break;
            case 2: action_search(root);    break;
            case 3: action_update(root);    break;
            case 4: action_delete(&root);   break;
            case 5: action_marksheet(root); break;
            case 6: action_stats(root);     break;
            case 7: { char path[128]; read_name("  CSV path: ", path); load_csv(&root, path); break; }
            case 8: { char path[128]; read_name("  CSV path: ", path); save_csv(root, path); break; }
            case 0: {
                int n = bst_count(root);
                bst_free(root); root = NULL;
```

```
                printf("  freed %d nodes - all memory released. Goodbye.\n", n);
                return 0;
            }
            default: printf("  ! unknown choice\n");
        }
    }
}
```

Appendix B - Source code: benchmark.c

```
/*
 * Benchmark suite for the Academic Records Indexer.
 *
 * Compares, for N = 1 000 ... 1 000 000 simulated student records:
 *   (a) BST lookup                      - O(log N) average
 *   (b) linear search in an unsorted array - O(N)
 *   (c) binary search in a sorted array   - O(log N)   (fair comparison)
 * It also demonstrates the BST worst case: keys inserted in sorted order
 * degenerate the tree into a linked list (height = N).
 *
 * Timing uses clock() from <time.h>. Because a single BST lookup takes well
 * under clock()'s resolution, each method is repeated until at least
 * MIN_SECONDS of CPU time has elapsed and the average per lookup is reported.
 * Comparison counts are exact and independent of the machine.
 *
 * Build:  gcc -O2 -Wall -o benchmark benchmark.c
 * Run:    ./benchmark            (prints a table, writes benchmark_results.csv)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>


#define MIN_SECONDS 0.25
#define LOOKUPS     1000          /* distinct random keys looked up          */
#define BASE_ROLL   2026100000L   /* roll numbers look like real ones        */


typedef struct Node {
    long roll;
    float cia1, cia2, end_sem;
    int is_eligible, has_arrears;
    struct Node *left, *right;
} Node;


static long long g_cmp;            /* global comparison counter */


/* ---------- BST ---------- */
static Node *new_node(long roll)
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (!n) { fprintf(stderr, "out of memory\n"); exit(1); }
    n->roll = roll; n->cia1 = n->cia2 = n->end_sem = 0;
    n->is_eligible = 1; n->has_arrears = 0;
    n->left = n->right = NULL;
    return n;
}
static void bst_insert(Node **root, long roll)
{
    Node **cur = root;
    while (*cur) cur = (roll < (*cur)->roll) ? &(*cur)->left : &(*cur)->right;
    *cur = new_node(roll);
}
static Node *bst_search(Node *root, long roll)
{
    while (root) {
        g_cmp++;
        if (roll == root->roll) return root;
        root = (roll < root->roll) ? root->left : root->right;
    }
    return NULL;
}
/* iterative height (safe even for the degenerate 10 000-deep tree) */
static int bst_height(Node *root)
{
    if (!root) return 0;
    size_t cap = 1024, top = 0; int best = 0;
    Node **st = malloc(cap * sizeof *st); int *dp = malloc(cap * sizeof *dp);
    st[top] = root; dp[top] = 1; top++;
    while (top) {
        top--; Node *n = st[top]; int d = dp[top];
        if (d > best) best = d;
        Node *kids[2] = { n->left, n->right };
        for (int i = 0; i < 2; i++) if (kids[i]) {
            if (top == cap) { cap *= 2; st = realloc(st, cap * sizeof *st); dp = realloc(dp, cap * sizeof *dp); }
            st[top] = kids[i]; dp[top] = d + 1; top++;
        }
    }
    free(st); free(dp);
    return best;
}
/* iterative post-order free */
static void bst_free(Node *root)
```

```
{
    if (!root) return;
    size_t cap = 1024, top = 0;
    Node **st = malloc(cap * sizeof *st);
    st[top++] = root;
    while (top) {
        Node *n = st[--top];
        if (n->left)  { if (top == cap) { cap *= 2; st = realloc(st, cap * sizeof *st); } st[top++] = n->left; }
        if (n->right) { if (top == cap) { cap *= 2; st = realloc(st, cap * sizeof *st); } st[top++] = n->right; }
        free(n);
    }
    free(st);
}

/* ---------- arrays ---------- */
static int linear_search(const long *a, long n, long key)
{
    for (long i = 0; i < n; i++) { g_cmp++; if (a[i] == key) return 1; }
    return 0;
}
static int binary_search(const long *a, long n, long key)
{
    long lo = 0, hi = n - 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        g_cmp++;
        if (a[mid] == key) return 1;
        if (key < a[mid]) hi = mid - 1; else lo = mid + 1;
    }
    return 0;
}
static int cmp_long(const void *x, const void *y)
{
    long a = *(const long *)x, b = *(const long *)y;
    return (a > b) - (a < b);
}

/* ---------- timing helper: returns average seconds per lookup ---------- */
typedef enum { M_BST, M_LINEAR, M_BINARY } Method;
static double time_lookups(Method m, Node *root, const long *arr, long n,
                           const long *keys, int nkeys, double *avg_cmp)
{
    long long total_lookups = 0; g_cmp = 0;
    clock_t start = clock(); double elapsed;
    volatile int sink = 0;
    do {
        for (int i = 0; i < nkeys; i++) {
            if (m == M_BST)         sink += (bst_search(root, keys[i]) != NULL);
            else if (m == M_LINEAR) sink += linear_search(arr, n, keys[i]);
            else                    sink += binary_search(arr, n, keys[i]);
        }
        total_lookups += nkeys;
        elapsed = (double)(clock() - start) / CLOCKS_PER_SEC;
    } while (elapsed < MIN_SECONDS);
    (void)sink;
    *avg_cmp = (double)g_cmp / (double)total_lookups;
    return elapsed / (double)total_lookups;
}

static void shuffle(long *a, long n)
{
    for (long i = n - 1; i > 0; i--) {
        long j = (long)(((unsigned long)rand() * 32768UL + (unsigned long)rand()) % (unsigned long)(i + 1));
        long t = a[i]; a[i] = a[j]; a[j] = t;
    }
}

int main(void)
{
    const long sizes[] = { 1000, 10000, 100000, 1000000 };
    const int nsizes = sizeof sizes / sizeof sizes[0];
    srand(42);                                  /* reproducible */
    FILE *csv = fopen("benchmark_results.csv", "w");
    if (csv) fprintf(csv, "N,bst_height,bst_build_ms,bst_lookup_us,bst_cmp,linear_lookup_us,linear_cmp,binary_lookup_
us,binary_cmp,speedup_bst_vs_linear\n");

    printf("Academic Records Indexer - lookup benchmark (clock() from <time.h>, %d random keys per round)\n\n", LOOKU
PS);
    printf("%9s | %6s | %9s | %12s %8s | %13s %10s | %13s %8s | %8s\n",
           "N", "height", "build ms", "BST us/lkp", "cmp", "Linear us/lkp", "cmp", "Binary us/lkp", "cmp", "speed-
up");
    printf("----------+--------+-----------+----------------------+--------------------------+----------------------
--+---------\n");
```

```
    for (int s = 0; s < nsizes; s++) {
        long n = sizes[s];
        long *keys_all = malloc(n * sizeof(long));
        for (long i = 0; i < n; i++) keys_all[i] = BASE_ROLL + i;
        shuffle(keys_all, n);                   /* random insertion order */


        /* build BST */
        clock_t t0 = clock();
        Node *root = NULL;
        for (long i = 0; i < n; i++) bst_insert(&root, keys_all[i]);
        double build_ms = 1000.0 * (double)(clock() - t0) / CLOCKS_PER_SEC;
        int height = bst_height(root);


        /* arrays: unsorted copy for linear search, sorted copy for binary search */
        long *unsorted = malloc(n * sizeof(long)); memcpy(unsorted, keys_all, n * sizeof(long));
        long *sorted   = malloc(n * sizeof(long)); memcpy(sorted,   keys_all, n * sizeof(long));
        qsort(sorted, n, sizeof(long), cmp_long);


        /* random existing keys to look up */
        long keys[LOOKUPS];
        for (int i = 0; i < LOOKUPS; i++) keys[i] = keys_all[(long)(((unsigned long)rand() * 32768UL + (unsigned long
)rand()) % (unsigned long)n)];


        double c_bst, c_lin, c_bin;
        double t_bst = time_lookups(M_BST,    root, NULL,     n, keys, LOOKUPS, &c_bst);
        double t_lin = time_lookups(M_LINEAR, NULL, unsorted, n, keys, LOOKUPS, &c_lin);
        double t_bin = time_lookups(M_BINARY, NULL, sorted,   n, keys, LOOKUPS, &c_bin);


        printf("%9ld | %6d | %9.1f | %12.3f %8.1f | %13.3f %10.1f | %13.3f %8.1f | %7.0fx\n",
               n, height, build_ms, t_bst * 1e6, c_bst, t_lin * 1e6, c_lin, t_bin * 1e6, c_bin, t_lin / t_bst);
        if (csv) fprintf(csv, "%ld,%d,%.3f,%.4f,%.2f,%.4f,%.2f,%.4f,%.2f,%.1f\n",
                         n, height, build_ms, t_bst * 1e6, c_bst, t_lin * 1e6, c_lin, t_bin * 1e6, c_bin, t_lin / t_b
st);
        fflush(stdout);


        bst_free(root); free(unsorted); free(sorted); free(keys_all);
    }


    /* ---- worst case: sorted insertion order degenerates the BST ---- */
    {
        long n = 10000;
        Node *root = NULL;
        for (long i = 0; i < n; i++) bst_insert(&root, BASE_ROLL + i);   /* ascending order */
        long keys[LOOKUPS];
        for (int i = 0; i < LOOKUPS; i++) keys[i] = BASE_ROLL + (long)(((unsigned long)rand() * 32768UL + (unsigned l
ong)rand()) % (unsigned long)n);
        double c; double t = time_lookups(M_BST, root, NULL, n, keys, LOOKUPS, &c);
        printf("\nWorst case - %ld keys inserted in SORTED order: height = %d (a linked list),"
               " avg %.1f comparisons, %.3f us per lookup  -> O(N); fix = AVL / Red-Black tree\n",
               n, bst_height(root), c, t * 1e6);
        if (csv) fprintf(csv, "# sorted-
insert worst case N=%ld height=%d cmp=%.2f us=%.4f\n", n, bst_height(root), c, t * 1e6);
        bst_free(root);
    }
    if (csv) { fclose(csv); printf("\nresults written to benchmark_results.csv\n"); }
    return 0;
}
```

Appendix C - Sample data (data/students.csv)

```
# roll_number,name,cia1,cia2,end_sem   (CIA out of 50, end-sem out of 100)
# insertion order is deliberately scrambled so the BST does not degenerate
2026166020,Karthik Raman,38,41,72
2026166008,Priya Lakshmi,45,47,88
2026166031,Aravind Selvam,29,33,55
2026166014,Meenakshi Sundaram,42,40,81
2026166026,Rahul Verma,15,18,40
2026166003,Anjali Nair,47,49,93
2026166037,Vignesh Kumar,33,36,61
2026166011,Divya Bharathi,40,38,70
2026166028,Deeptanshu Kumar,44,46,86
2026166017,Sneha Iyer,36,34,58
2026166035,Harish Babu,22,25,48
2026166005,Fathima Begum,41,44,79
2026166023,Gokul Krishnan,30,28,52
2026166039,Lavanya Devi,48,46,90
2026166001,Abdul Rahman,37,39,66
2026166012,Nithya Shree,43,45,84
2026166030,Suresh Pandian,26,31,44
2026166019,Pooja Reddy,39,42,75
2026166033,Manoj Prabhu,12,16,35
2026166007,Keerthana M,46,48,91
2026166024,Ramya Krishnan,34,37,63
2026166016,Bharath Raj,31,29,57
2026166038,Swathi Mohan,42,43,77
2026166002,Ajay Menon,35,33,69
2026166040,Yuvraj Singh,40,41,74
```